Practical: Data Analysis & Vectors

Instructor: Dr. Khaled Mohamad

> **Instructions:** Answer all questions in order. Write clean, commented code. Use Markdown cells for written answers.

In [1]:
```
# import all necessary lib
```

Q1. Distance Matrix with NumPy Broadcasting

You have **3 delivery locations** (group A) and **2 warehouses** (group B), given as 2D coordinates:

```
A = np.array([[1, 2],
              [4, 6],
              [7, 1]])

B = np.array([[0, 0],
              [5, 5]])
```

1. Reshape `A` and `B` for broadcasting: `A_exp = A[:, np.newaxis, :]` and `B_exp = B[np.newaxis, :, :]`. Print both shapes.
2. Compute the difference `diff = A_exp - B_exp`. Print its shape.
3. Compute the **Euclidean distance matrix** using `np.sqrt(np.sum(diff**2, axis=2))`. Print the result rounded to 2 decimal places.
4. In a Markdown cell, answer: which warehouse (B0 or B1) is **closer on average** to all delivery locations?

ANSWER

In [ ]:

```

```

In [2]:

```
# 1) reshape for broadcasting
```

In [3]:

```
#2) difference
```

In [4]:

```
#3) distance matrix
```

In [5]:

```
#4) average distance per warehouse
```

Answer Q1.4: ...

Q2. Given:

```
p = [2, 1, -3]
q = [4, -2, 1]
```

1. Compute `p · q` using four different methods (loop, `np.dot`, `np.multiply + sum`, `np.matmul`).
2. Using the **geometric formula** `p · q = ||p|| × ||q|| × cos(θ)`, compute the angle θ between `p` and `q` in **both radians and degrees**.
3. Interpret the angle: are the vectors more aligned, perpendicular, or opposing?
4. Now create a vector `r` that is **guaranteed to be orthogonal to `p`** (dot product = 0) in 3D. Verify it numerically.

ANSWER

In [ ]:

```

```

In [6]:

```
# 1) four methods
```

In [7]:

```
# 2) angle
```

In [8]:

```
# 3) interpretation
```

In [9]:
```
# 4) construct orthogonal vector
```

Q3. Given vectors:

```
u = [3, -4]
v = [-6, 8]
```

1. Compute the magnitudes of `u` and `v`. What is the ratio `||v|| / ||u||`? What does this ratio tell you?
2. Compute the unit vectors of `u` and `v`. What do you notice?
3. Are `u` and `v` parallel? Justify your answer mathematically (do **not** just plot — use the dot product and magnitudes).
4. On a single plot, draw `u`, `v`, and the unit vector of `u`. Add a legend and dashed axes.

ANSWER

In [ ]:
```

```

In [10]:
```
# 1) magnitudes and ratio
```

In [11]:
```
# 2) unit vectors
```

In [12]:
```
# 3) parallel check using dot product
```

In [13]:
```
# 4) Plot
```

Q4. Pandas: Load, Explore & Clean

Load the **MPG** dataset (car fuel efficiency), a dataset built into Seaborn:

```
df = sns.load_dataset('mpg')
```

Each row is a car model. Key columns: `mpg` (miles per gallon), `cylinders`, `weight`, `horsepower`, `origin`, `name`.

1. Print the **shape** and display the **first 5 rows**.
2. Print the **number of missing values per column**. Which column has missing values?
3. Create `df_clean` by **dropping rows** with any missing values using `.dropna()`. Print the new shape.
4. Print basic **summary statistics** using `.describe()`. What is the average `mpg`?

ANSWER

In [14]:
```
# load the dataset
```

In [15]:
```
# 1)
```

In [16]:
```
#2)
```

In [17]:
```
# 3)
```

In [18]:
```
# 4)
```

Your answers:

- ...
- ...

Q5. Filtering, Sorting & GroupBy

Using `df_clean` from Q2:

1. **Filter** cars with `mpg > 30` (fuel-efficient cars). Print how many there are.
2. **Sort** the dataset by `mpg` in **descending order** and display the **top 5** most fuel-efficient cars — show only the columns `name`, `mpg`, `cylinders`, `origin`.
3. Use **GroupBy** to find the **average mpg per country of origin** (`origin`). Print the result.
4. In one Markdown sentence, state which country produces the most fuel-efficient cars on average.

ANSWER

In [19]:
```
# 1)
```

In [20]:
```
# 2)
```

In [21]:
```
# 3)
```

Answer Q3.4: ...

Q6. Visualization

Using `df_clean`, create a figure with **two subplots side by side** (`figsize=(12, 4)`):

- **Left subplot:** A **bar plot** of average `mpg` per `origin`. Title: `'Average MPG by Country'`. Add a y-axis label `'MPG'`.
- **Right subplot:** A **scatter plot** of `weight` (x-axis) vs `mpg` (y-axis). Title: `'MPG vs Car Weight'`. Add axis labels.

Use `plt.tight_layout()` and `plt.show()`. In a Markdown cell, write **one observation** from the scatter plot.

ANSWER

In [ ]:
```

```

Observation: ...

Q7. KNN Classification on the Wine Dataset

The **Wine dataset** contains chemical measurements of wines from 3 different cultivars (classes 0, 1, 2). It is loaded from sklearn.

1. Load the dataset and print `feature_names`, `target_names`, and the shape of the data.

2. Split into **train/test sets** (75% train, 25% test) using `random_state=42`.
3. Train a `KNeighborsClassifier` with `n_neighbors=5`. Print the **test accuracy**.
4. In a Markdown cell, state in one sentence whether this is a **classification** or **regression** problem and why.

ANSWER

In [22]:
```
from sklearn.datasets import load_wine
# 1) load
```

In [23]:
```
# 2) split
```

In [24]:
```
# 3) train and evaluate
```

Answer Q7.4: ...

In [ ]:
```

```